

Async FIFO — Design & Simulation Documentation

Dual-Clock · Gray-Code Pointers · 2-FF CDC Synchronizers · Depth 8 · 8-bit Wide

Tools: Icarus Verilog · GTKWave · ModelSim · Vivado

1. Module Hierarchy

```
fifo_top
|-- wr_hdlr    (wr_hdlr)    write pointer + full flag  [wclk domain]
|-- rd_hdlr    (rd_hdlr)    read pointer  + empty flag  [rclk domain]
|-- fifo_mem           8x8 dual-port SRAM
|-- ffsynchor (ffsynchor)  g_rptr --> wclk  (2-FF CDC sync)
`-- ffsynchow (ffsynchow)  g_wptr --> rclk  (2-FF CDC sync)
```

2. RTL Schematic Notes

Each ffsynchor instance synthesizes to two **RTL_REG_ASYNC** primitives (C/CLR/D/Q) in series, confirming the 2-stage flip-flop synchronizer. Internal nodes: **out_reg_reg[3:0]** (stage 1) and **dout_reg[3:0]** (stage 2).

Signal	From	To	Purpose
g_wptr[3:0]	wr_hdlr	ffsynchow.din	Gray write pointer (wclk domain)
g_wptr_sync[3:0]	ffsynchow	rd_hdlr	g_wptr synchronized to rclk
g_rptr[3:0]	rd_hdlr	ffsynchor.din	Gray read pointer (rclk domain)
g_rptr_sync[3:0]	ffsynchor	wr_hdlr	g_rptr synchronized to wclk
b_wptr[3:0]	wr_hdlr	fifo_mem.b_wptr	Binary write address
b_rptr[3:0]	rd_hdlr	fifo_mem.b_rptr	Binary read address
data_in[7:0]	top port	fifo_mem	Write data
data_out[7:0]	fifo_mem	top port	Read data (asynchronous)
full	wr_hdlr	top + fifo_mem	FIFO full flag
empty	rd_hdlr	top port	FIFO empty flag

3. Core Logic Per Module

wr_hdlr — Write Handler [wclk domain]

```
b_wptr_next = b_wptr + (w_en & !full);
g_wptr_next = (b_wptr_next >> 1) ^ b_wptr_next; // binary to Gray
wfull       = (g_wptr_next == {~g_rptr_sync[3:2], g_rptr_sync[1:0]});
```

Full: Write pointer has wrapped one extra time relative to read pointer. In Gray space, upper 2 bits invert; lower bits match the synced read pointer.

rd_hdlr — Read Handler [rclk domain]

```
b_rptr_next = b_rptr + (r_en & !empty);
g_rptr_next = (b_rptr_next >> 1) ^ b_rptr_next;
rempty      = (g_wptr_sync == g_rptr_next); // plain equality
```

Empty: Both pointers at same position in same wrap. No bit inversion needed — symmetric opposite of full detection.

ffsynchor — 2-FF Synchronizer (instantiated x2)

```
always @(posedge clk or negedge rst) begin
    out_reg <= din; // Stage 1: captures (may be metastable)
    dout    <= out_reg; // Stage 2: resolved output
end
```

Safe only on single-bit-change signals — guaranteed by Gray code encoding. One full clock period is allowed for metastability resolution between the two stages.

Instance	clk	rst	din	dout	Purpose
ffsynchor	wclk	wrst_n	g_rptr	g_rptr_sync	Read pointer to write domain
ffsynchrow	rclk	rrst_n	g_wptr	g_wptr_sync	Write pointer to read domain

fifo_mem — 8x8 Dual-Port SRAM

```
always @(posedge wclk) if (w_en & !full) mem[b_wptr[2:0]] <= data_in;
assign data_out = mem[b_rptr[2:0]]; // asynchronous combinatorial read
```

Write is synchronous to wclk. Read is combinatorial (b_rptr already registered in rd_hndlr). Memory contents are not reset — treated as don't-care until first write.

4. Simulation — Testbench Setup

Parameter	Value
wclk period	10 ns (100 MHz)
rclk period	25 ns (40 MHz)
Clock ratio	wclk 2.5x faster than rclk
FIFO depth	8 entries
Data width	8 bits
Pointer width	4 bits (N+1, where N=3)

Test Phases

1. Reset	(0-30 ns)	wrst_n=0, rrst_n=0 => all ptrs=0, empty=1, full=0
2. Write	(~50 ns+)	loop: w_en=1, random data_in => until full=1
3. Read	(after gap)	loop: r_en=1 => until empty=1
4. Concurrent	(15 cycles)	w_en=1, r_en=1 simultaneously, data_in++

5. Waveform — Key Observations

Write Phase (~50–270 ns)

b_wptr increments 0 to 8 each wclk cycle; g_wptr follows Gray encoding. g_wptr_sync (read domain) lags g_wptr by 2 rclk cycles (~50 ns). data_in sequence: **24 81 09 63 0D 8D 65 12 01**. full=1 asserted after the 8th write.

Read Phase (~320–500 ns)

b_rptr increments each rclk cycle. data_out sequence: **24 81 09 63 0D 8D 65 12 24** — correct FIFO order. empty=1 after all 8 entries are read. g_rptr_sync in write domain updates with 2-wclk lag (~20 ns).

Concurrent R/W Phase (~550–700 ns)

data_in increments sequentially: **02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10**. data_out follows with read-domain latency. full and empty remain deasserted — steady-state throughput achieved.

CDC Synchronizer Lag

Crossing	Lag
g_wptr → g_wptr_sync	2 rclk cycles ≈ 50 ns
g_rptr → g_rptr_sync	2 wclk cycles ≈ 20 ns

6. Waveform Signal Groups (ModelSim / GTKWave)

Group	Signals
States	clk, rst
READ_HANDLER	rrst_n, rclk, r_en, g_wptr_sync[3:0], g_rptr[3:0], b_rptr[3:0], empty
WRITE_HANDLER	wrst_n, wclk, w_en, g_rptr_sync[3:0], g_wptr[3:0], b_wptr[3:0], full
Top-level	wclk, w_en, b_wptr[3:0], data_in[7:0], b_rptr[3:0], data_out[7:0]
SYNCHRONIZE_READ	clk, din[3:0], out_reg[3:0], dout[3:0] (ffsynchror internals)
synchronize_write	clk, din[3:0], out_reg[3:0], dout[3:0] (ffsynchrow internals)

7. References

1. Cummings, C. E. — *Simulation and Synthesis Techniques for Asynchronous FIFO Design*, SNUG 2002

2. Cummings, C. E. — *...with Asynchronous Pointer Comparisons*, SNUG 2002

3. Spear, C. — *SystemVerilog for Verification*, Springer, 3rd ed.